Memory optimization for small language model security agents

CISM is an autonomous vulnerability assessment agent driven by a model small enough to run on the operator's own machine.

Madhan T B	23BCE1087
Manikandan S	23BCE1476
Sushindh A	23BCE1513

VELLORE INSTITUTE OF TECHNOLOGY, CHENNAI



context used against the window

Headroom is not the answer. A three hour engagement will spend whatever window it is given.

THE PROBLEM

A 96k window still fills inside one engagement.

The server reports a window of 98304 tokens. The agent reads that number at startup rather than assuming one, so the budget is the real window.

One engagement reached 76014 tokens. That is past 0.75 of the budget, which is where compaction starts working whether or not anything has overflowed yet.

MOTIVATION

Why a small model on a 6 GB laptop.

Three constraints pick the design. None of them is a preference.

CONSTRAINT 01

The engagement data cannot leave the building.

A hosted model strong enough for this work needs target names, banners, credentials and raw tool output sent to a third party. The contract usually forbids exactly that, so the model has to run on the operator's own machine.

CONSTRAINT 02

The machine is a laptop, not a cluster.

The driver has to fit a 6 GB consumer GPU alongside Windows. Nothing much above 9B runs at a usable speed there. The model will be weak at exactly the judgement this task needs.

THE TURN

The obvious fix is the wrong one.

A security tuned model looks like the answer. We tested that directly: every community cybersecurity fine-tune except one full merge got worse at calling tools than the model it was built from. Domain knowledge belongs in the agent's tools, not in the weights.

RTX 3050	24 GB	90
6 GB LAPTOP GPU	SYSTEM RAM	CALLS PER MODEL

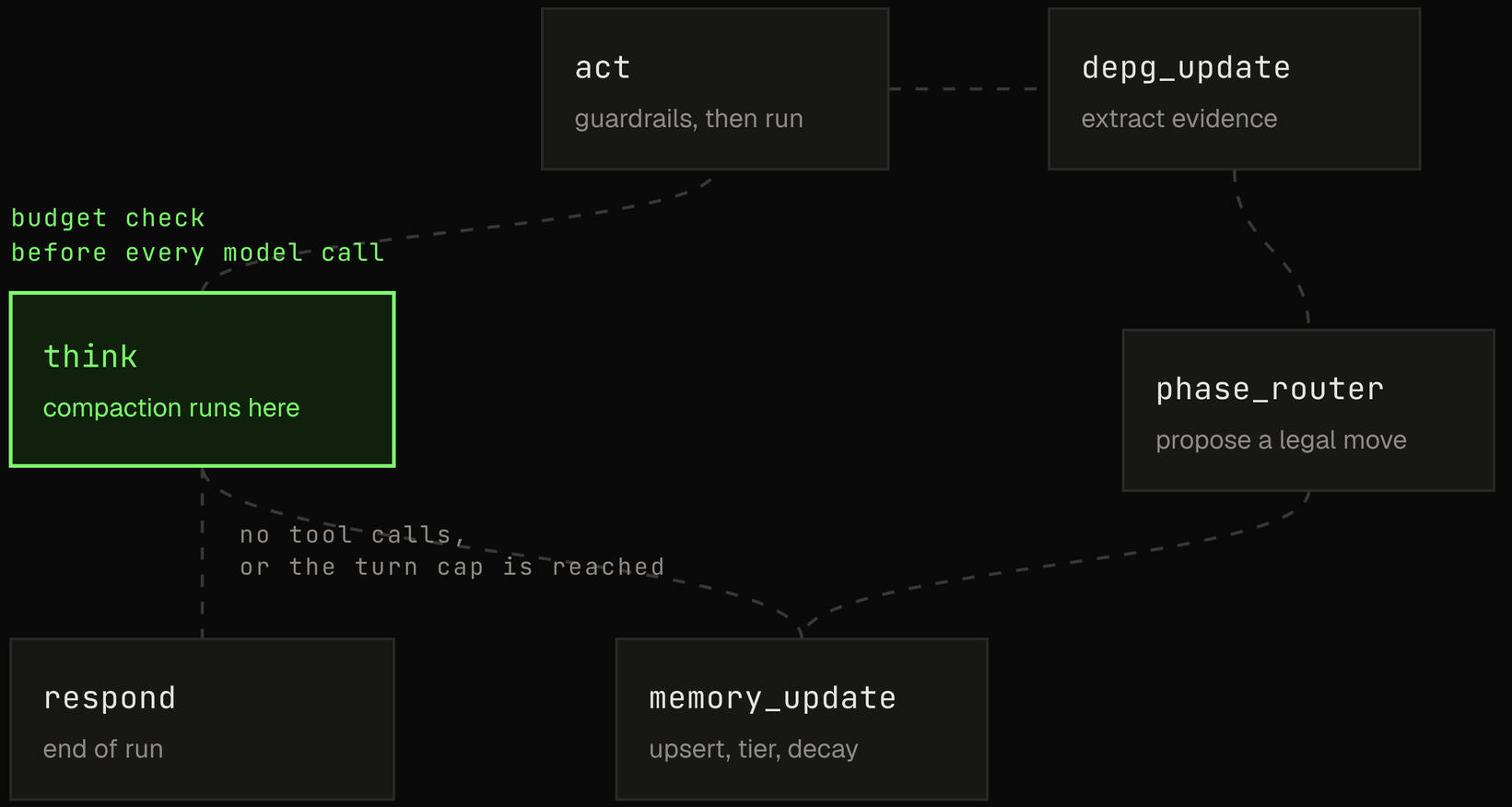
Architecture

01

The model supplies direction. Memory, grounding, phase discipline and pre execution policy come from the system around it.

THE AGENT LOOP

Six nodes, one loop, one budget check.



Click any node.

The agent is a LangGraph state machine with six nodes: think, act, extract, route, remember and respond.

State persists across submissions within one process through a checkpointer.

WHERE COMPACTION TRIGGERS

Compaction runs inside think, before the model is called.

```
_CHARS_PER_TOKEN = 3
_RESPONSE_RESERVE_TOKENS = 512
_PER_MESSAGE_OVERHEAD_TOKENS = 4
DEFAULT_CONTEXT_WINDOW = 32_768

def _compact_history(messages, system_prompt, context_window):
    budget = (context_window
              - _RESPONSE_RESERVE_TOKENS
              - _estimate_tokens(system_prompt))

    kept, dropped = _fit(messages)
    if dropped:
        kept, dropped = _fit(_shrink_old_tool_output(messages))
    if dropped:
        kept, dropped = _fit(_stub_old_tool_output(messages))
    return kept, dropped
```

The budget

No generic local tokenizer exists, so the estimate is 3 characters per token with 4 tokens of overhead per message. 512 tokens stay reserved for the answer.

The window

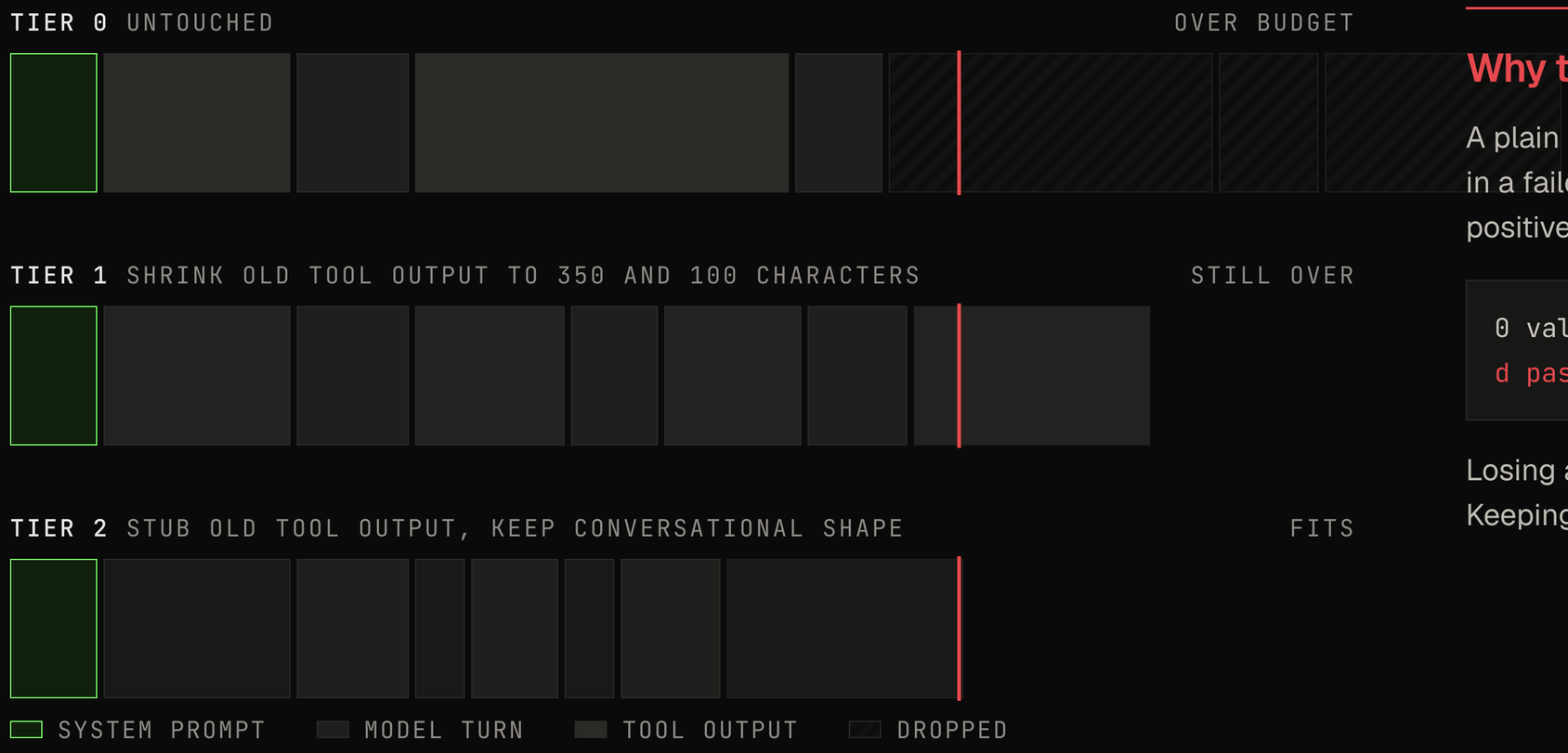
The default window is 32768 tokens. The system prompt is subtracted first, because it is sent on every turn.

The cascade

Each tier runs only when the one before it still did not fit. Full history stays in state. This only shapes what is sent to the model.

THE CASCADE

Three tiers, cheapest sufficient wins.



A tier engages when the one before it could not fit the budget, or when usage is already past 0.75 of it.

Why the cut snaps to a word

A plain character cut landed inside the word valid in a failed hydra run. The tail it left read as a positive hit.

```
0 valid password found
d password found
```

Losing a partial word at either edge costs nothing. Keeping half of one can flip what a finding means.

PHASE LOCK

An illegal phase jump is not reachable.

a jump from recon straight to exploit is refused by transition()



The audit found the router advancing one phase per successful tool call, reaching reporting in about 5 calls. The table is what made that visible.

Six phases, one choke point.

Every phase change goes through `PhaseStateMachine.transition()`. Nothing else may move the phase.

Dashed arcs are the legal pivots back. Reporting accepts only itself.

GUARDRAILS

Nothing spawns before every stage allows it.

01	`ScopeGuardStage`	Blocks a call whose target falls outside the declared engagement scope, a CIDR and domain allowlist.
02	`DangerousFlagGuardStage`	Blocks free form flags that could read or write outside the scratch directory, or trigger code execution.
03	`PhaseToolGuardStage`	Re-checks the tool against the current phase allowlist at call time, not at registration time.
04	`ToolOutputScreenStage`	Screens tool output for prompt injection markers before it is fed back into the model.
05	`OutputRedactionStage`	Redacts credentials and secrets from the final answer the operator sees and exports.

> The catalog fails closed. A tool that writes, executes or destroys is registered but unreachable in every phase until someone adds an explicit phases key.

CONTAINMENT

One argv only seam carries every real tool call.



WHAT THE SEAM GUARANTEES

No shell

Execution is argv only. A quoted string never reaches an interpreter.

No persistence

The per call scratch directory no longer exists once the call returns.

No orphans

A hard timeout kills the whole process tree, not just the leaf.

No arbitrary paths

hydra's wordlist arguments only ever select one of two fixed, named wordlists.

2 GB MEMORY	2 GB SWAP
2 PROCESSORS	512 PROCESSES

Memory

02

A claim the model cannot ground against real tool text is demoted instead of reported.

DEPENDENCY AND ERROR PROPAGATION GRAPH

Every finding carries the output that proves it.

illustrative graph shape

T1 evidence, quoted from gobuster



$$r = s_o \cdot (1 - \prod(1 - w \cdot r_v))$$

≥ 0.6 resolved

≤ 0.15 blocked

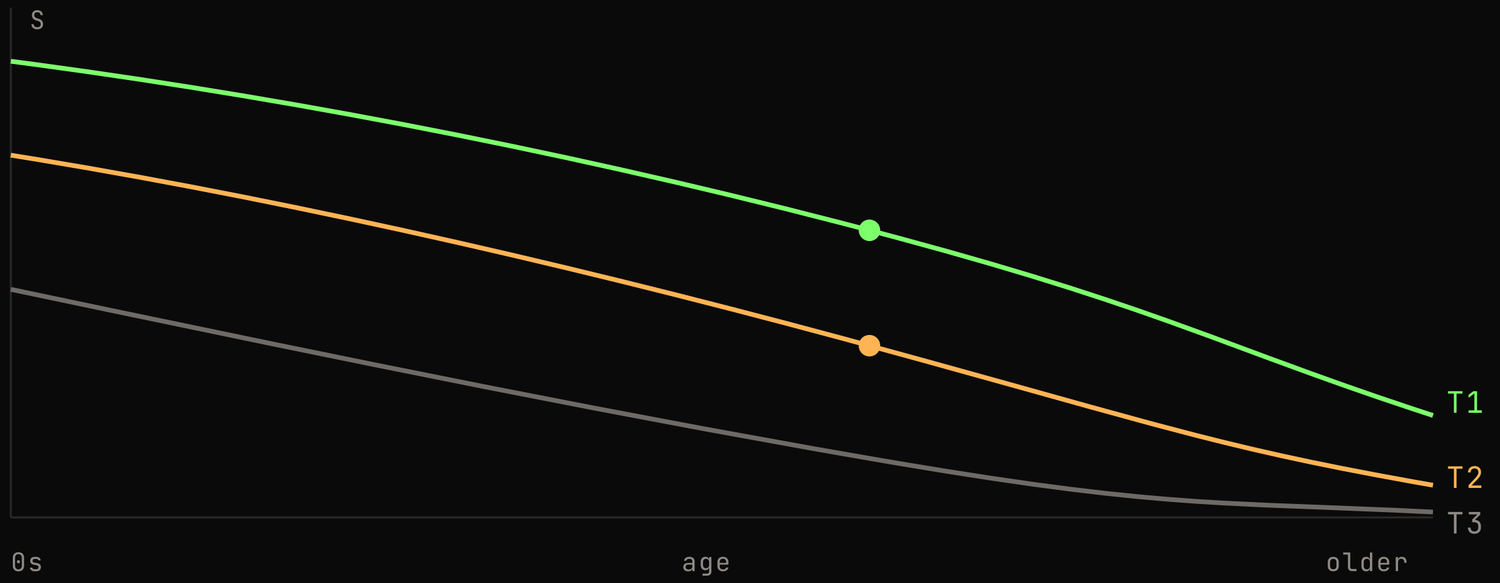
Click any node.

Each edge carries a weight, the propagation coefficient. Each node carries r, its reachability.

The graph merges proposed nodes, resolves each dirty node, then prunes fully dead regions.

TIERED MEMORY

Memory decays unless the run keeps hitting it.



Each item carries a tier, the phase it was seen in, a snippet and a salience score.

A recurring narration or tool failure collapses onto one row and increments its hit count instead of filling the view.

Tier weight sets the starting height. The hit bonus is what lifts a row back up.

$$S = w_{tier} \cdot s_0 \cdot e^{(-\lambda \cdot age)} + \beta \cdot hits$$

Results

03

Measured against a containerized Damn Vulnerable Web Application on an isolated bridge network, with its database initialized.

EXPERIMENTAL SETUP

Real targets, real tools, real local inference.

TARGETS

Damn Vulnerable Web Application on an isolated bridge network, at 172.28.0.2 and 172.28.0.3, database initialized. Scope is those addresses alone.

DRIVER

gemma-4-E4B-it-Q4_K_M served by llama-server at 98304 context, fully offloaded to the GPU.

AGENT

Inside the hardened container. All capabilities dropped except the one bubblewrap requires.

INSTRUCTIONS

Three scripted submits covering reconnaissance, enumeration and injection testing.

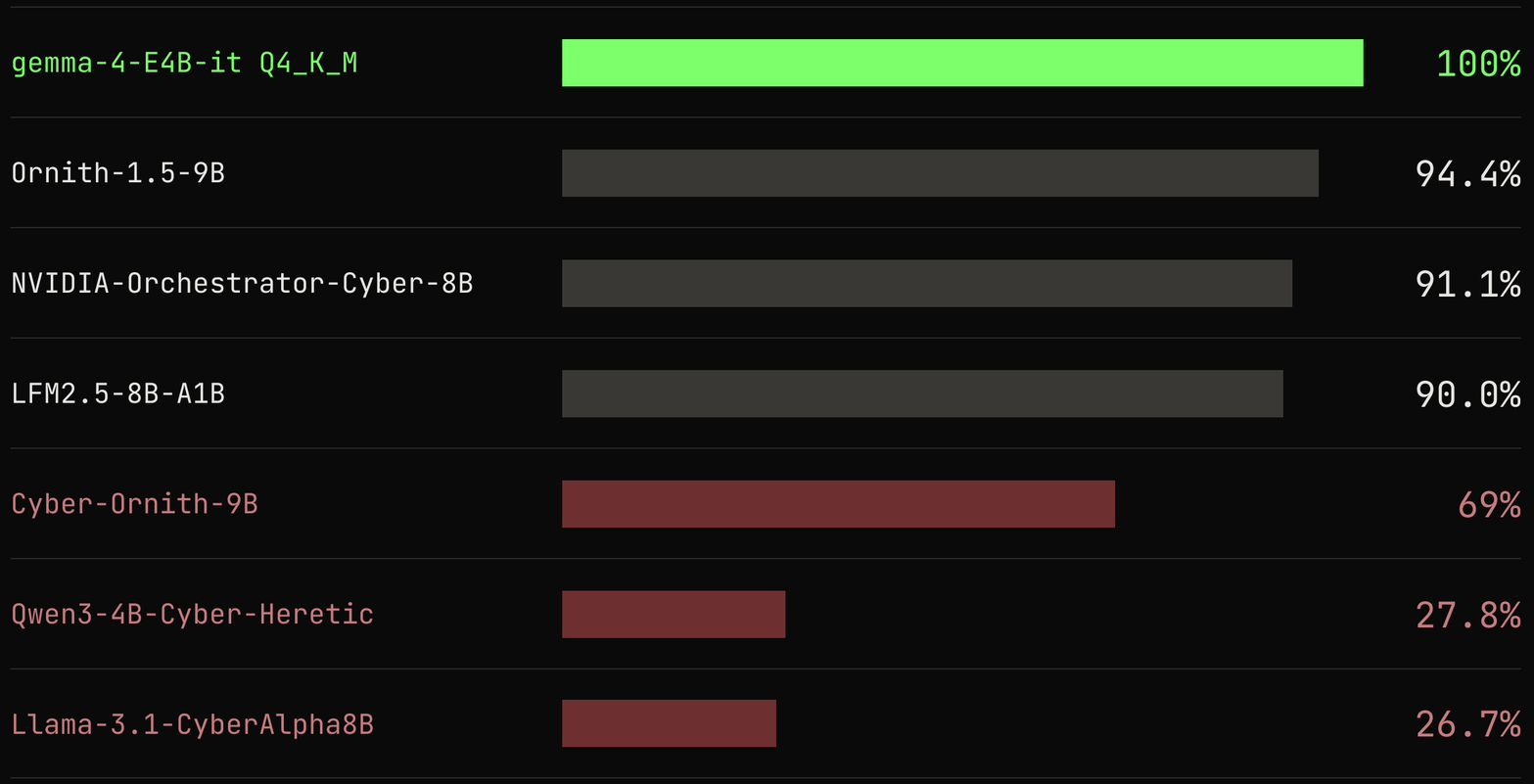
CEILINGS

2 GB

memory and swap, 2 processors, 512 processes.

MODEL SELECTION

Every security fine-tune we tested got worse at calling tools.



18 case types, 5 repeats, 90 calls per model at temperature 0.2. Correct tool, correct argument types, and correctly declining when no tool applies.

The red rows are the security models.

Cyber-Ornith is Ornith with security tuning on top. It lost 25 points to the model it was built from.

These fine-tunes train on security question and answer text, not on function calling, so the skill erodes.

Domain knowledge belongs in the agent's tools, not in the weights. That finding is what the rest of this project is built on.

CONTEXT BUDGET

96k fits in 6 GB because of Flash Attention.



Full GPU offload, four bit weights and a fused attention kernel are what hold this curve down.

98304 tokens costs 5.23 GB and leaves about 900 MB of headroom.

131072 was measured and rejected. It leaves roughly 350 MB, which is not enough to survive anything else touching the card.

0 → 8

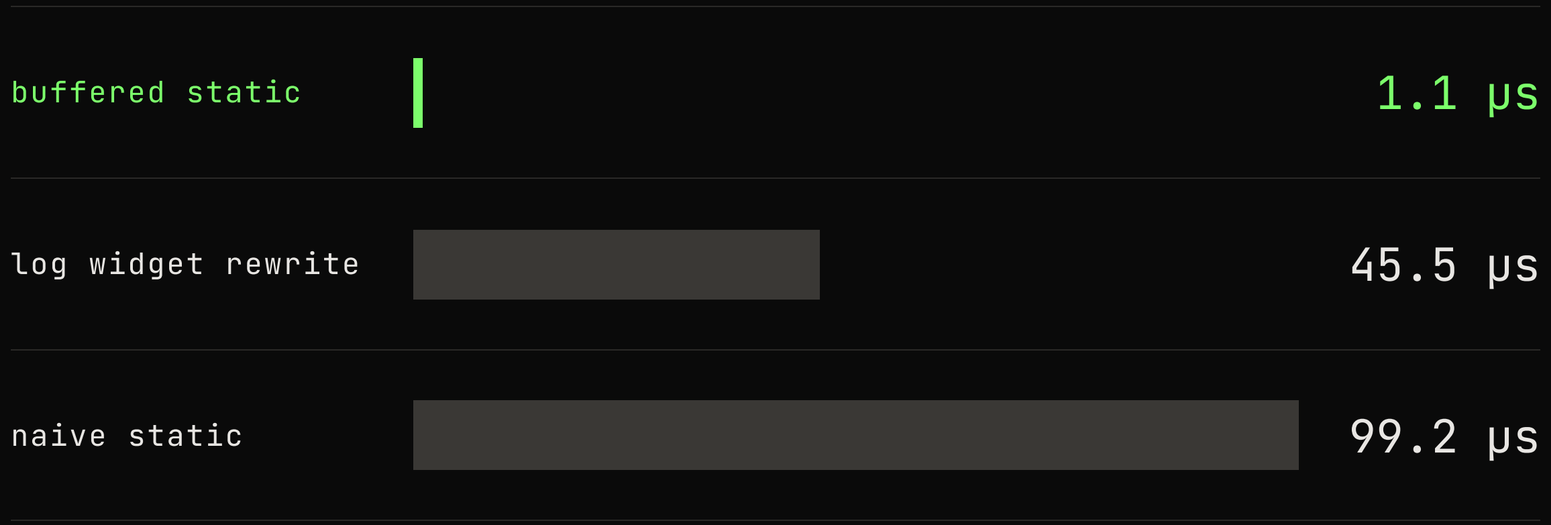
Correcting three extractor defects took one engagement from 0 to 8 evidence backed findings. Same model, same target, same instructions.

TOOL	BEFORE	AFTER	DEFECT
gobuster	0	5	progress escapes defeated the pattern
whatweb	0	2	no extractor existed
wafw00f	0	1	assumed a secure scheme
total	**0**	**8**	**correctness, not a new capability**

All three defects were invisible to a test suite written against clean sample output. All three were visible at once against real output from a real target.

RENDERING BENCHMARK

The shipped transcript stays flat as the entry grows.



The benchmark streams 8000 tokens into one wrapping paragraph in a terminal 100 columns by 30 rows.

The shipped strategy measures 1.4 microseconds at 4000 tokens and 1.1 at 8000. The naive strategy climbs from 58 to 99 over the same range, which is quadratic in entry length.

per token cost / lower is better / bench/stream_bench.py

VALIDATION

786 tests and a harness repeatable to a tenth of a second.

786

automated tests across 53 modules, mirroring the source tree roughly one to one. Ten of them are terminal snapshots that pin the rendered appearance.

53

test modules

6500

lines of code in the audited package, read line by line

HEADLESS HARNESS

RUN 1	RUN 2
2.9 s	3.0 s

time to first finding, 1 finding each

10

snapshot tests pinning the terminal appearance

OBSERVED FAILURE MODES

The model lies about its own evidence.

FAILURE 01

It reported a success that the response body flatly contradicts

The injection never reached the SQL parser. The body came back byte for byte as the normal result: `First name: admin`. The narration said the attack was successful and confirmed the database version, then planned the next step on that premise.

FAILURE 02

It forgot a correction it had just been given

The same encoding mistake came back across four separate turns. The correct form sat in its own immediately preceding successful call.

FAILURE 03

Corrected once, it over-generalized

Told to use a literal separator instead of encoding it, it stopped escaping anything at all, and curl rejected the URL outright.

> Neither model found or exploited an actual SQL injection in its observed window. We are reporting that, because it is the result.

LIMITS AND NEXT PHASE

Every limit names the seam it extends into.

WHAT THE WORK DOES NOT CLAIM

Evaluation covered two targets on one application. It does not generalize to other applications.

No confirmed SQL injection exploit landed in either model's observed window.

Checkpointed state does not survive a process restart.

The sandbox shares the host network, so scope is enforced on argument text and not at the operating system boundary.

FUTURE WORK, HIGHEST IMPACT FIRST

- → Make a discarded model attempt invisible to the event stream.
- → Add an argument validation guardrail stage.
- → Replace the denylist with an allowlist grammar per tool.
- → Implement network egress control for the sandbox.

The architecture carries the model.

0 to 8

findings from one engagement after three extraction defects were corrected, with no change to the model.

786

automated tests, plus a headless harness repeatable to a tenth of a second.

10

real tools behind one sandboxed, argv only seam and a guardrail chain that runs first.